A Quantum Algorithm For String Matching

May 26, 2026

Problem Definition

Given:

Text:

$$T = t_0 t_1 t_2 \cdots t_{N-1}$$

Pattern:

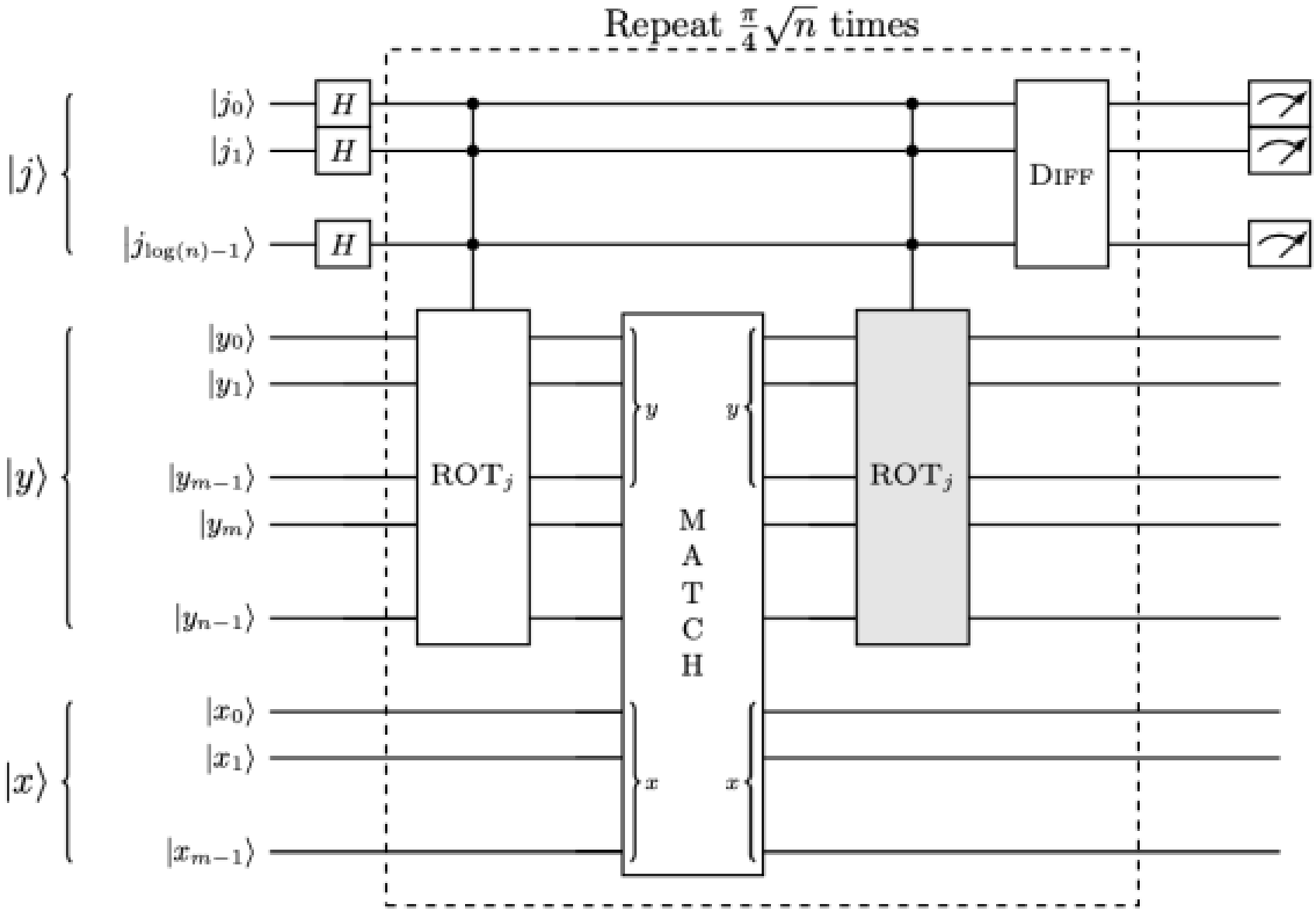
$$P = p_0 p_1 p_2 \cdots p_{M-1}$$

Goal:

$$t_k t_{k+1} \cdots t_{k+M-1} = p_0 p_1 \cdots p_{M-1}$$

Find the index k where the pattern appears in the text.

Complete Quantum String Matching Circuit



Quantum State Encoding

$$|T\rangle = |t_0 t_1 \cdots t_{N-1}\rangle = \bigotimes_{i=0}^{N-1} |t_i\rangle$$

Text state: Stores all bits/characters of the input text in qubits. This register will later undergo cyclic shifts.

$$|P\rangle = |p_0 p_1 \cdots p_{M-1}\rangle = \bigotimes_{j=0}^{M-1} |p_j\rangle$$

Pattern state: Stores the pattern to be searched and will be compared with shifted text.

$$|\psi\rangle = |0\rangle^{\otimes n} \otimes \left(\bigotimes_{i=0}^{N-1} |t_i\rangle \right) \otimes \left(\bigotimes_{j=0}^{M-1} |p_j\rangle \right)$$

Initial quantum state: Combines index register, text register, and pattern register. The index register starts in $|0\rangle^{\otimes n}$ and later enters superposition using Hadamard gates.

Note: $N = 2^n$

Step 2: Create Superposition

Apply Hadamard gates on the index register:

$$H^{\otimes n}$$

Transformation:

$$|0\rangle \rightarrow \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} |j\rangle$$

Resulting quantum state:

$$|\psi_1\rangle = \left(\frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} |k\rangle \right) \left[\bigotimes_{i=0}^{N-1} |t_i\rangle \right] \left[\bigotimes_{j=0}^{M-1} |p_j\rangle \right] \tag{5}$$

- Create all candidate positions simultaneously

Step 3: Apply Cyclic Shift Operator

Apply cyclic shift operator S:

S left-circular shifts the text qubits by k positions, where k is encoded in the index register.

Applying S on the first two registers gives:

$$\left[S\left(\frac{1}{\sqrt{N}}\sum_{k=0}^{N-1}|k\rangle\right)\left(\bigotimes_{i=0}^{N-1}|t_i\rangle\right)\right]\left(\bigotimes_{j=0}^{M-1}|p_j\rangle\right)$$

$$=\frac{1}{\sqrt{N}}\sum_{k=0}^{N-1}|k\rangle\left(\bigotimes_{i=0}^{N-1}|t_{i+k}\rangle\right)\left(\bigotimes_{j=0}^{M-1}|p_j\rangle\right) \tag{6}$$

Purpose:

- Align text with all possible pattern positions
- Prepare shifted text states for comparison
- Perform all shifts simultaneously

Construction of Cyclic Shift Operator

Cyclic-shift operator rotates qubits by s positions:

$$|q_0, q_1, \ldots, q_{N-1}\rangle \rightarrow |q_s, q_{s+1}, \ldots, q_{N-1}, q_0, q_1, \ldots, q_{s-1}\rangle$$

Applying shift operator S on text state:

$$S|k\rangle \left(\bigotimes_{i=0}^{N-1} |t_i\rangle \right) = |k\rangle \left(\bigotimes_{i=0}^{N-1} |t_{i+k}\rangle \right)$$

Binary encoding of shift index:

$$|k\rangle = |k_0 k_1 \cdots k_{n-1}\rangle$$

where

$$k = 2^0 k_0 + 2^1 k_1 + \cdots + 2^{n-1} k_{n-1}$$

Binary Decomposition of Shift Operator

- Instead of building a separate circuit for every shift, use:

$$S^a S^b = S^{a+b}$$

- Express shift index k in binary form:

$$k = 2^0 k_0 + 2^1 k_1 + \cdots + 2^{n-1} k_{n-1}$$

- Construct the shift operator:

$$S_k = S_{2^0}^{k_0} S_{2^1}^{k_1} \cdots S_{2^{n-1}}^{k_{n-1}}$$

- Example:

$$13 = (1101)_2$$

$$13 = 2^3 + 2^2 + 2^0$$

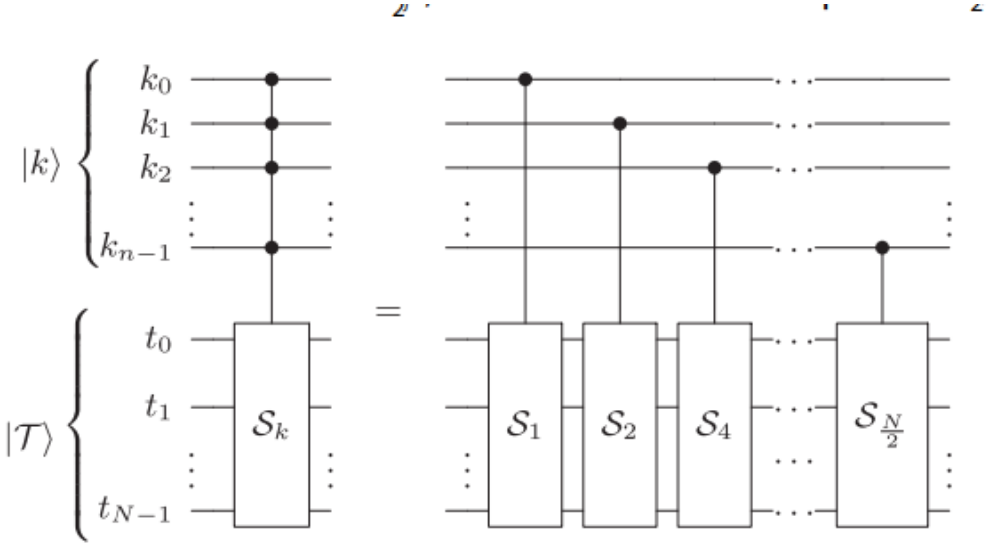
$$S_{13} = S_8 S_4 S_1$$

Controlled Construction of Shift Operator

$$|k\rangle \left(\bigotimes_{i=0}^{N-1} S_k |t_i\rangle \right) = \left(\bigotimes_{j=0}^{n-1} |k_j\rangle \right) \left(\bigotimes_{i=0}^{N-1} \left(\prod_{j=0}^{n-1} S_{2^j}^{(k_j)} \right) |t_i\rangle \right)$$

where $S_{2^j}^{(k_j)}$ applies a shift of 2^j bits on the text register.

- If $k_j = 1$, apply shift S_{2^j}
- If $k_j = 0$, skip that shift
- Combining active shifts generates total shift k



Step 5: Shift Implementation

Shift permutation:

$$P_s = \{N - s,\ N - s + 1,\ \ldots,\ N - s - 1\}$$

Maps qubit positions to implement a circular shift of s bits.

Implemented using:

$$SWAP(q_i, q_j)$$

SWAP exchanges the positions of two qubits. Multiple SWAP operations together realize the required permutation.

Parallel layers:

- Layer 1 : $N/2$ swaps
- Layer 2 : $N/4$ swaps
- Layer 3 : $N/8$ swaps
- $\ldots$

The number of swaps reduces by half at each layer because many swaps can execute simultaneously.

Depth per shift layer:

$$O(\log N)$$

Since the number of layers is logarithmic.

Total shift depth:

$$O((\log N)^2)$$

There are $\log N$ binary-controlled shifts and each requires $O(\log N)$ depth.

Step 4: XOR Comparison Using CNOT

Apply XOR between:

- First M bits of shifted text register
- M bits of pattern register

CNOT gate implementation:

$$CNOT^M$$

Resulting state:

$$\frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} |k\rangle \left(\bigotimes_{i=0}^{N-1} |t_{i+k}\rangle \right) \left(\bigotimes_{j=0}^{M-1} |p_j \oplus t_{j+k}\rangle \right)$$

Interpretation:

- If

  $$p_j \oplus t_{j+k} = 0$$

  then bits match.
- If all XOR outputs are zero:

  $$|00\cdots0\rangle$$

  then

  $$P = T[k : k + M - 1]$$
- Otherwise nonzero bits indicate mismatches.

Step 6: Convert to All-Ones Problem

Apply X gates on XOR outputs:

$$a_i \to a_i \oplus 1$$

Effect:

$$0 \to 1 \qquad 1 \to 0$$

Interpretation:

- Equal bits $\to 1$
- Different bits $\to 0$

Purpose:

- Convert matching problem into an all-ones problem
- Simplify oracle implementation

Step 7: Multi-Controlled X Gate

Apply:

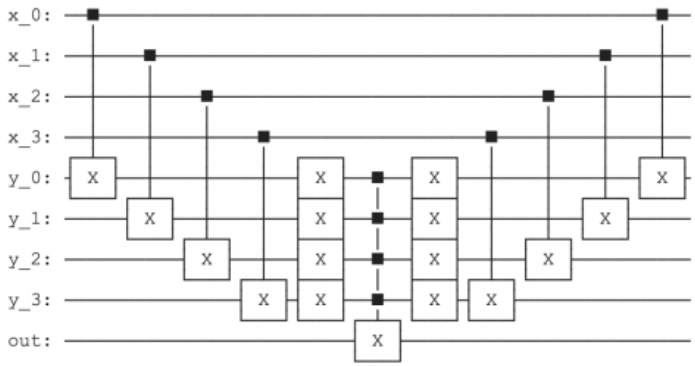
$$MCX$$

Operation:

$$|a_0 a_1 \cdots a_{M-1}\rangle|m\rangle$$

$$\rightarrow$$

$$|a_0 a_1 \cdots a_{M-1}\rangle|m \oplus (a_0 a_1 \cdots a_{M-1})\rangle$$



If

$$a_0 = a_1 = \cdots = a_{M-1} = 1$$

then

$$m = 1$$

Purpose:
Generate a match indicator bit

Step 8: Oracle Phase Flip

Prepare ancilla state:

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Apply controlled-X:

$$|m\rangle|-\rangle$$

Using:

$$X|-\rangle = -|-\rangle$$

Therefore:

$$|j\rangle|1\rangle|-\rangle$$

becomes

$$-|j\rangle|1\rangle|-\rangle$$

Oracle:

$$U_f|j\rangle = (-1)^{f(j)}|j\rangle$$

where

$$f(j) = \begin{cases} 1, & \text{match} \\ 0, & \text{otherwise} \end{cases}$$

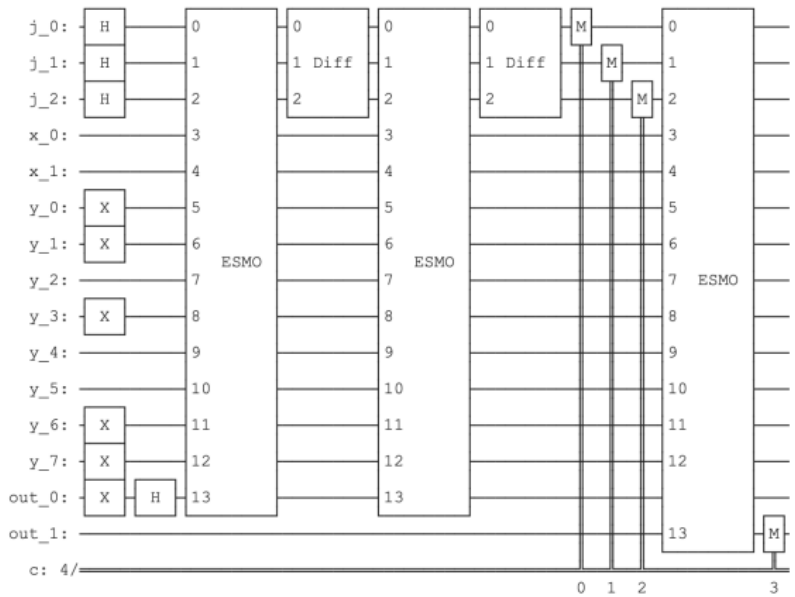
Step 10: Grover Diffusion Operator

Apply diffusion operator:

$$D = 2|u\rangle\langle u| - I$$

where

$$|u\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} |j\rangle$$



Search for 00 in 11010011

Step 11: Grover Iteration

Grover operator:

$$G = DU_f$$

Number of iterations:

$$R = \left\lfloor \frac{\pi}{4} \sqrt{\frac{N}{r}} \right\rfloor$$

where r is the number of matching positions.

Practical consideration:

- Number of matches r is generally unknown
- Exact value of R cannot be computed directly
- Use adaptive Grover search:

$$t = 1$$

$$t \to \frac{5}{4} t$$

- Randomly choose:

$$R \in \{0, 1, \ldots, t-1\}$$

- Repeat until a valid match index is obtained

Purpose:
Efficiently search even when the number of solutions is unknown.

Complexity Analysis

Component Complexity

Shift operator:

$$O((\log N)^2)$$

Grover Oracle:

$$O(\log M)$$

using

$$O(M)$$

ancilla qubits

Grover iterations:

$$O(\sqrt{N})$$

Space:

$$O(N + M)$$

Overall Complexity

Total time:

$$O\Big(\sqrt{N}\big((\log N)^2 + \log M\big)\Big)$$

Notes

- Oracle uses multi-controlled Toffoli gates
- Oracle depth:

  $$O(\log M)$$

- Requires:

  $$O(M)$$

  ancilla qubits
- Some architectures may support near constant-depth multi-control operations

Gate Count Analysis

Gate Cost per Component

Encoding:

$$0$$

(only X and Identity gates)

Hadamard:

$$\log(N)$$

H gates

Cyclic Shift:

$$(8N - 9)\log(N)$$

CNOT

$$7(N - 1)\log(N)$$

T gates

XOR:

$$M$$

CNOT gates

Oracle:

$$6M - 12$$

CNOT

$$8M - 17$$

Total Gate Count

$$\#CNOT = (7M - 12 + (8N - 9)\log N) \times 2\sqrt{N}$$

$$\#T = (8M - 17 + 7(N - 1)\log N) \times 2\sqrt{N}$$

Note

- Factor 2 appears because Grover uses:

  $$U \quad \text{and} \quad U^\dagger$$

- Repeat:

  $$O(\sqrt{N})$$

  times for amplitude amplification